

01

Lecture 01

Operating Systems Foundations

by - Brijesh Shukla
C07: Operating Systems

The Mystery

What happens when you click "Save"?



Course Structure

Two-Phase Approach:

Phase 1 (Weeks 1-8): OS Core Concepts + Projects

- Processes, scheduling, synchronization
- Memory management, I/O systems
- Deadlocks and coordination

Phase 2 (Weeks 9-12): Building an OS

- Nand2Tetris
- Jack programming language
- Implement Hack OS modules

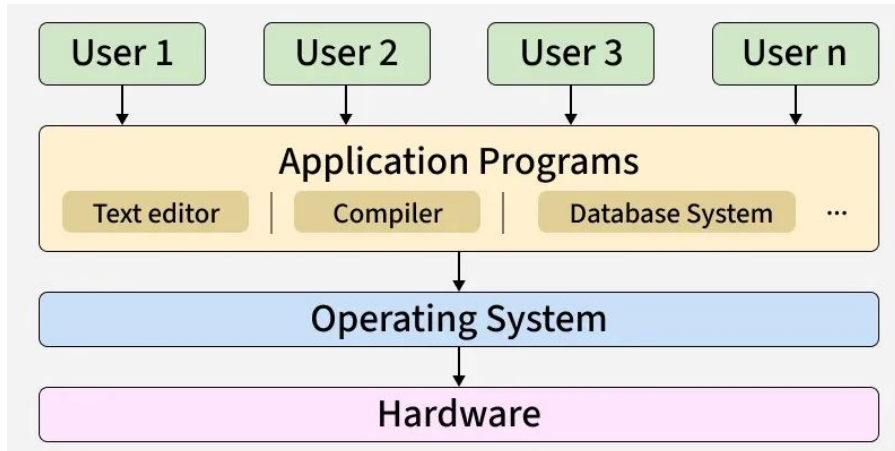


Why Study Operating Systems?

OS = Foundation of All Software

- **Performance:** Why some programs are fast/slow
- **Reliability:** Understanding crashes and failures
- **Security:** How systems protect resources
- **Career:** Systems knowledge in high demand

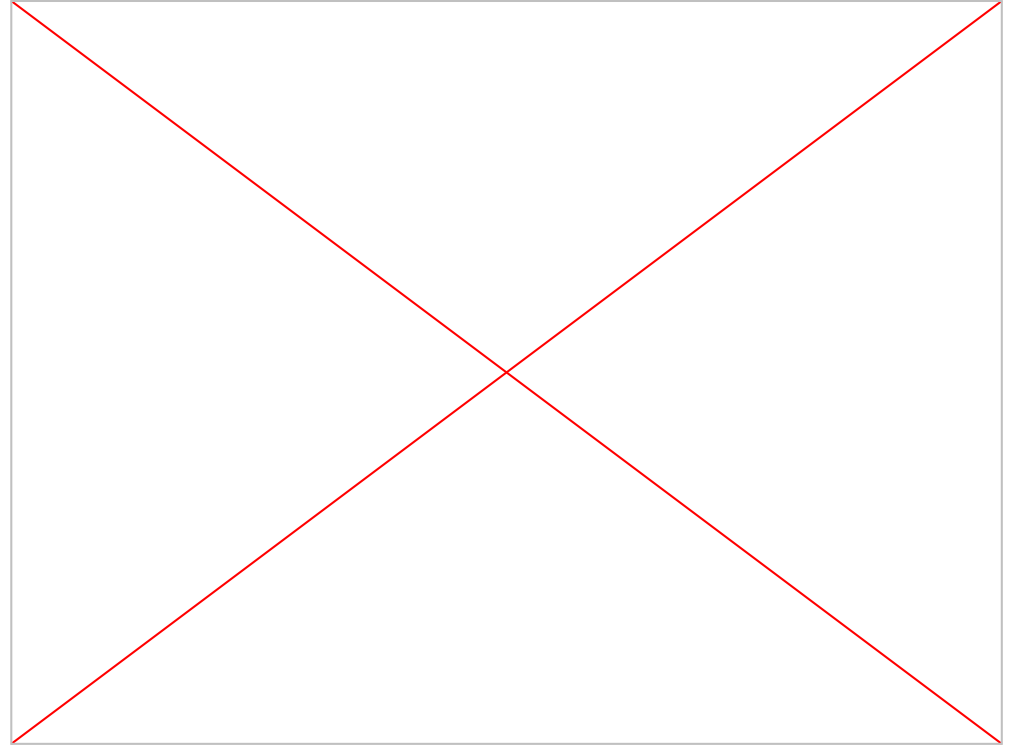
Every programmer benefits from OS understanding



What IS an Operating System?

Three Core Roles:

- **Resource Manager:** Allocates CPU, memory, I/O
- **Control Program:** Supervises execution, prevents errors
- **Extended Machine:** Simplifies hardware complexity



Analogy: OS is the conductor of your computer's orchestra

OS as City Infrastructure

Computer →

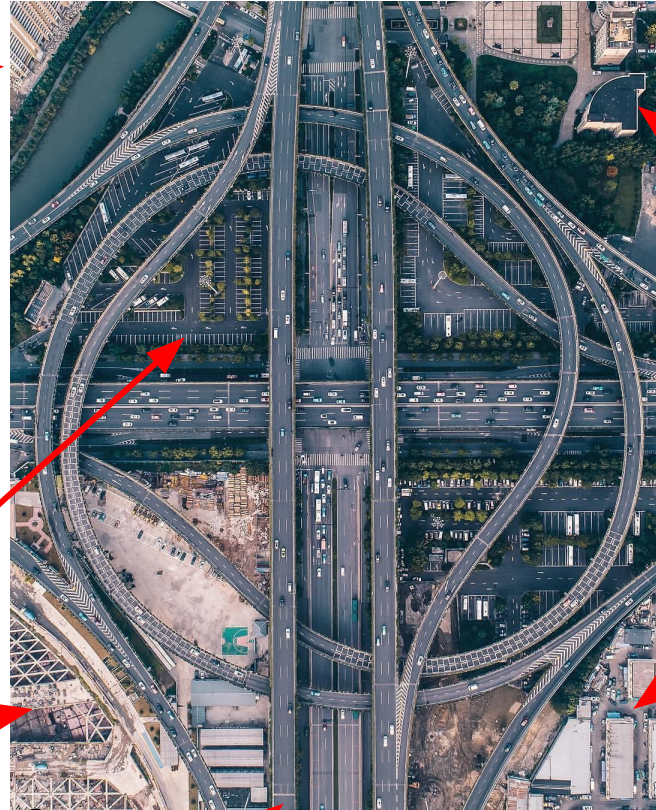
Computer = City, OS = Infrastructure

- Roads/Highways = System buses
- Power Grid = Power management
- Police/Courts = Security mechanisms
- Postal Service = Inter-process communication
- Zoning Board = Memory management

Without OS: Hardware is just expensive silicon

Memory
Management

Power
Management



Security

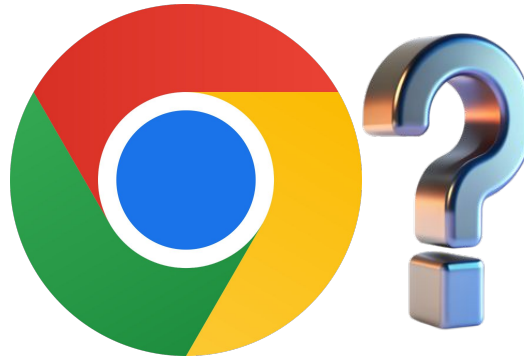
IPC

System
Bus

Checkpoint Question

Is a web browser an Operating System?

Think: Chrome manages processes, memory, security...



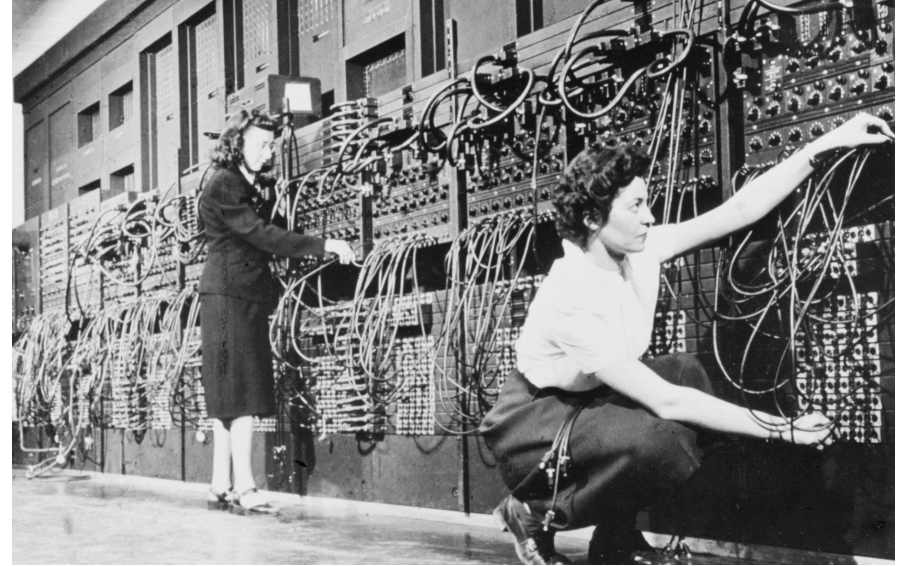
Answer: No - browsers use OS services, don't provide them

History – 1950s Batch Processing

The Dawn of Computing

- Programs on punch cards
- Manual job queues by operators
- No interaction during execution
- CPU idle most of the time

First OS: Automated batch processing monitors



History – 1960s Time-Sharing

The Revolution: Multics (Multiplexed Information and Computing Service)

- Multiple users, one computer simultaneously
- Rapid switching gives illusion of dedicated machine
- Introduced: memory protection, file permissions, virtual memory

Legacy: Concepts we still use today



History – 1970s–1980s UNIX Era

Two Parallel Tracks:

UNIX Evolution:

- UC Berkeley creates BSD
- Portability and simplicity
- macOS inherits BSD DNA

Personal Computing:

- Apple II, Commodore 64
- 1984: Macintosh brings GUI to masses

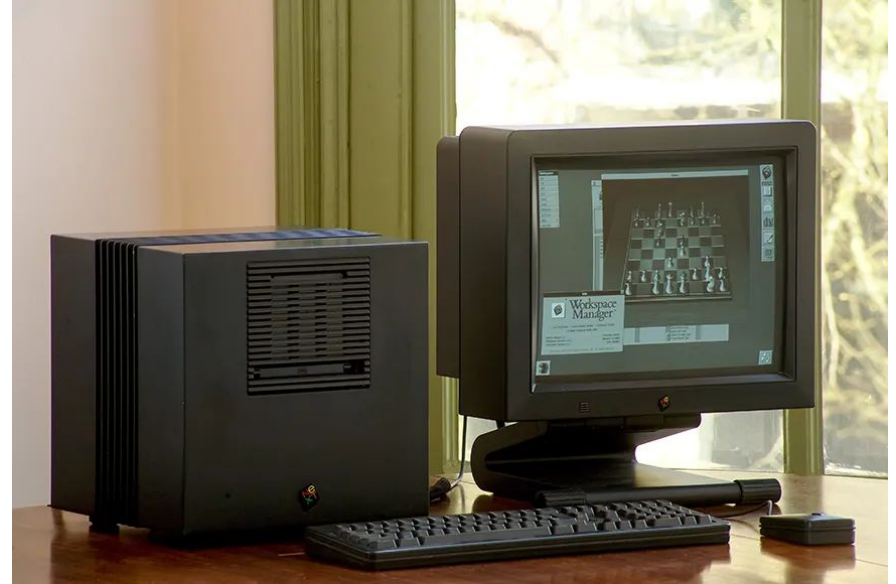


History – 1990s NeXTSTEP

Steve Jobs' NeXT Computer

- Mach microkernel + BSD UNIX
- Elegant object-oriented interface
- Tim Berners-Lee invented WWW on NeXT

1996: Apple acquires NeXT → foundation of modern macOS



History – Modern Era

2000s-Present:

- **2001:** Mac OS X launched (UNIX-certified)
- **2007:** iOS brings OS X to mobile
- **2017:** APFS replaces HFS+
- **2020:** Apple Silicon transition

Your Mac: Decades of OS evolution in one system

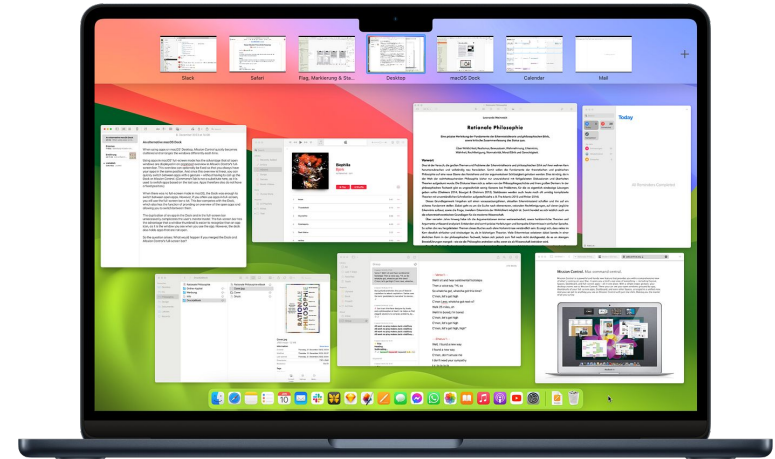


1 – Process Management

Managing Hundreds of Programs Simultaneously

- Process creation and termination (fork, exec, wait)
- CPU scheduling across cores
- Context switching
- Grand Central Dispatch on macOS

Key: Isolation + efficient resource sharing

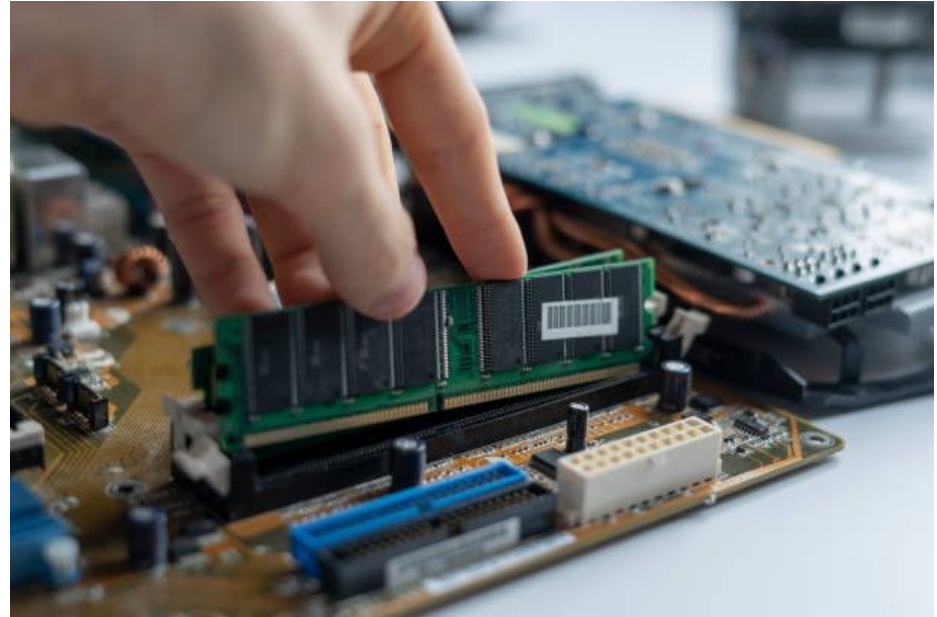


2 – Memory Management

The Infinite Memory Illusion

- Virtual memory per process
- Physical RAM sharing and protection
- Paging and swapping
- Memory compression on macOS

16GB RAM → Hundreds of processes using "gigabytes" each



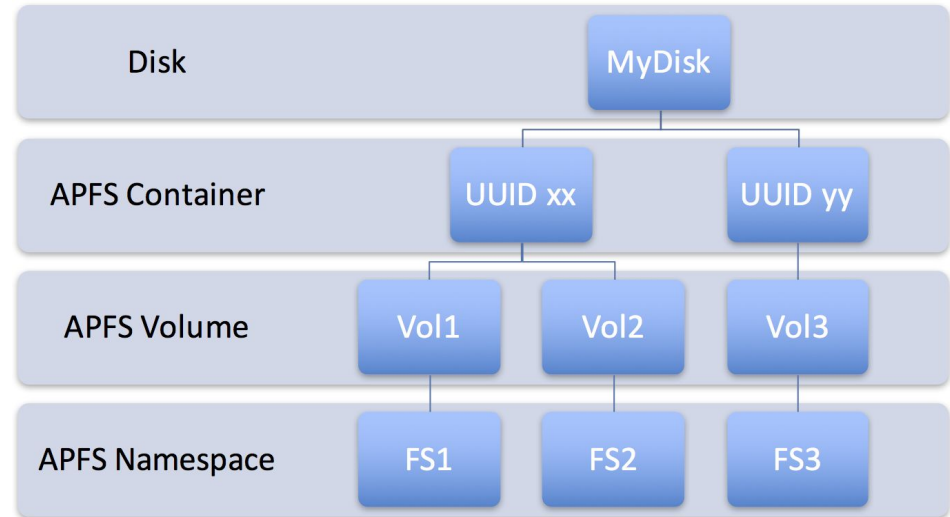
3 – File Systems

Organizing Persistent Storage

- APFS: Copy-on-write, snapshots, encryption
- Hierarchical directories
- Permissions and access control
- Metadata management

Billions of blocks → organized files you can find instantly

APFS

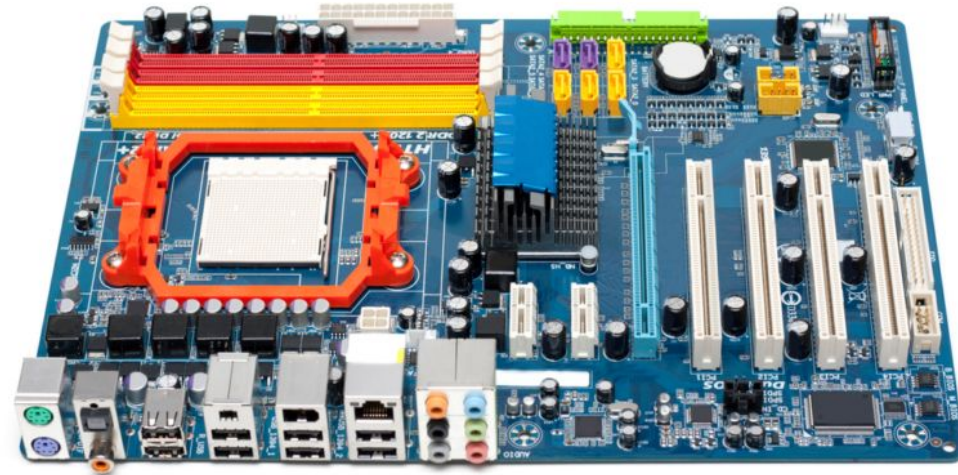


4 - I/O Management

Hardware Abstraction

- Device drivers (IOKit on macOS)
- Interrupts vs polling vs DMA
- Buffering and caching
- Hot plugging support

**Thousands of device types →
standardized interface**



5 – Security

Guardian of the System

- Process isolation (separate memory spaces)
- System Integrity Protection (SIP)
- Sandboxing (App Store apps)
- Gatekeeper and code signing
- Secure Enclave (Apple Silicon)

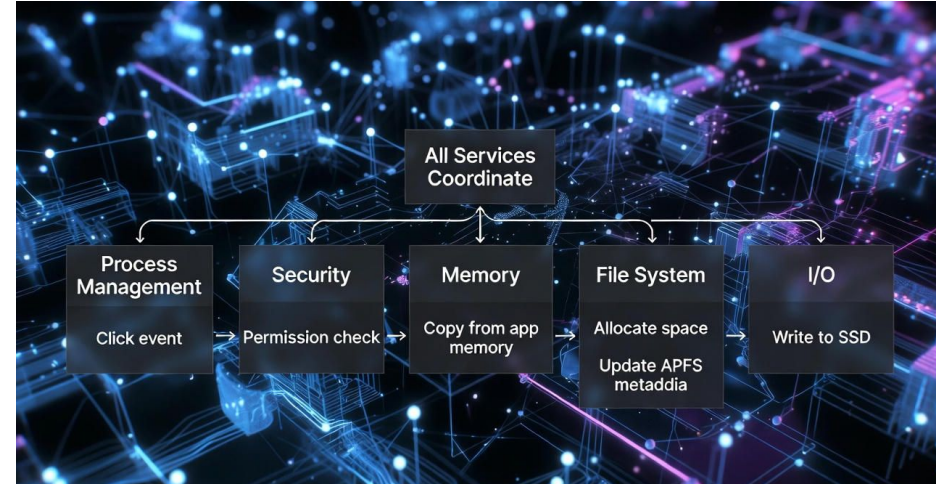
Security requires OS-level enforcement



Services Working Together

Saving a File: All Services Coordinate

1. Process Management: Handle click event
2. Security: Check permissions
3. Memory: Copy data from app memory
4. File System: Allocate space, update metadata (APFS)
5. I/O: Write to SSD
6. Return success to application



Simple action → complex orchestration

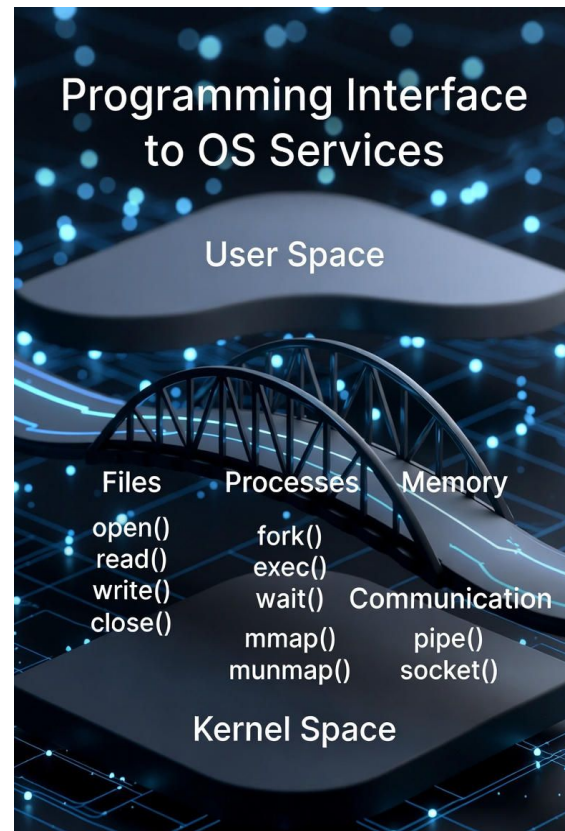
System Calls – The Bridge

Programming Interface to OS Services

Common System Calls:

- Files: `open()`, `read()`, `write()`, `close()`
- Processes: `fork()`, `exec()`, `wait()`
- Memory: `mmap()`, `munmap()`
- Communication: `pipe()`, `socket()`

POSIX standard → portable across UNIX systems

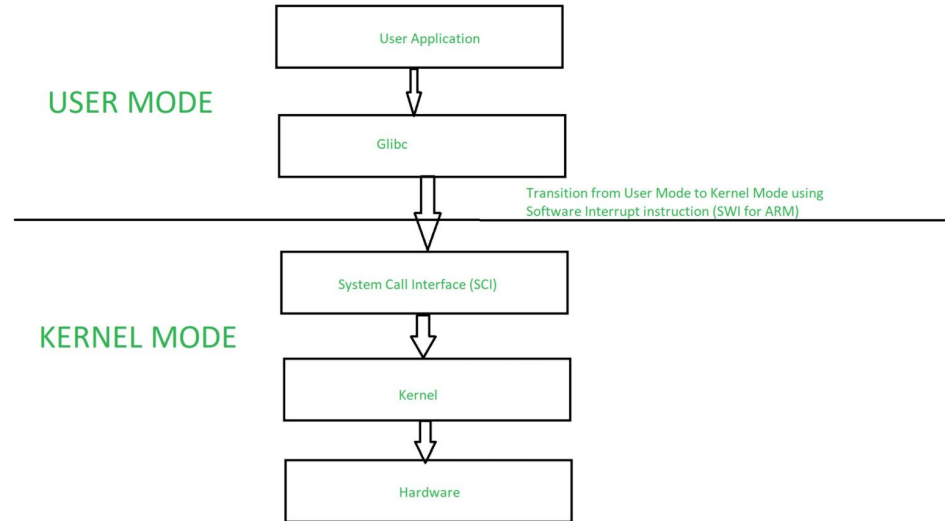


System Call Mechanism

From Application to Kernel:

1. App calls library function (e.g., write())
2. Load arguments into registers
3. Execute trap instruction
4. CPU switches to kernel mode
5. Kernel validates and executes
6. Return to user mode
7. Result returned to app

Cost: Microseconds per call (expensive in CPU time)



User Mode vs Kernel Mode

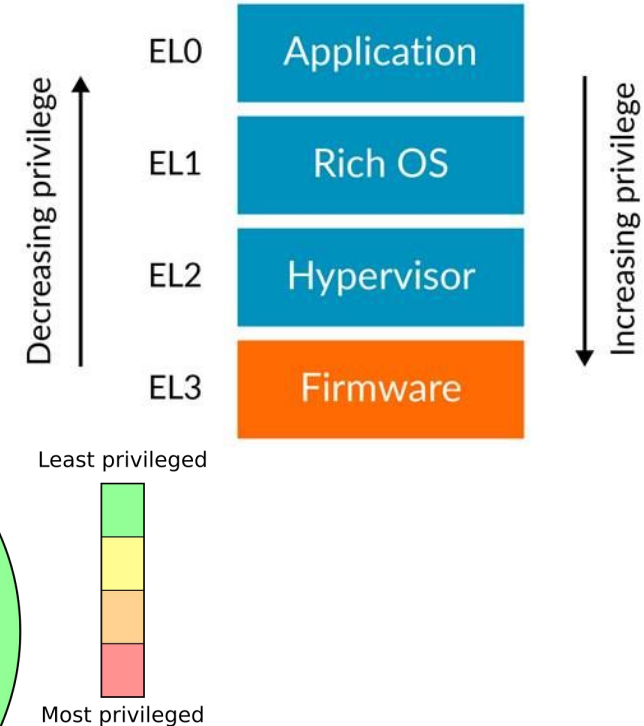
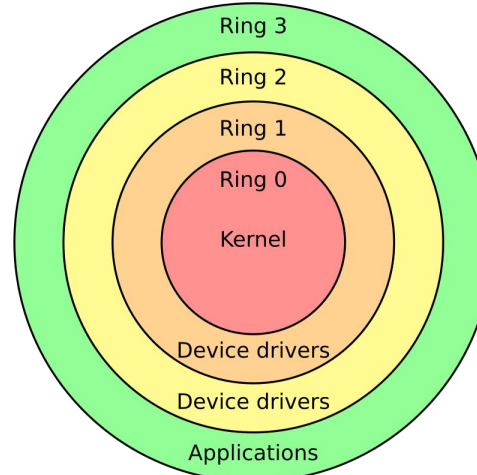
Two Privilege Levels

Apple Silicon (ARM64):

- EL0: User mode (your apps)
- EL1: Kernel mode (XNU)
- EL2: Hypervisor
- EL3: Secure Enclave

Why? Stability, security, abstraction

User mode cannot: Access hardware, other processes' memory, privileged instructions



Mode Transitions

Crossing the Privilege Boundary:

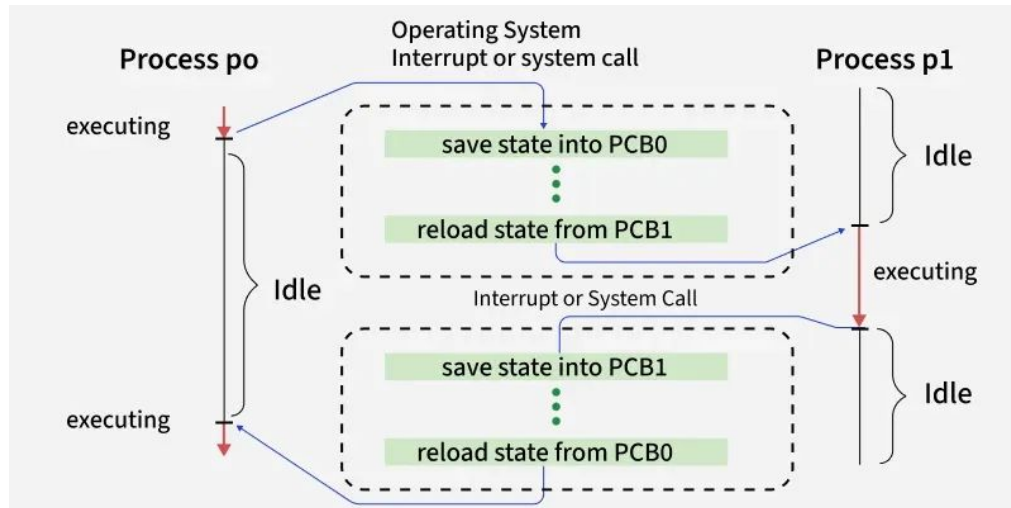
Triggers:

- System calls (intentional)
- Interrupts (hardware events)
- Exceptions (errors like page faults)

Process:

- Save user state
- Switch to kernel mode
- Execute kernel code
- Restore user state
- Return to user mode

Context switch: 2-5 microseconds overhead



Kernel Panic – When Protection Fails

Total System Failure

Causes:

- Defective hardware
- Kernel extension (KEXT) bugs
- Driver issues
- File system corruption

Why panic instead of recover?

Kernel state is suspect → continuing risks
data corruption

Modern macOS: Rare due to enhanced protections

You need to restart your computer. Hold down the Power button for several seconds or press the Restart button.

Veillez redémarrer votre ordinateur. Maintenez la touche de démarrage enfoncée pendant plusieurs secondes ou bien appuyez sur le bouton de réinitialisation.

Sie müssen Ihren Computer neu starten. Halten Sie dazu die Einschalttaste einige Sekunden gedrückt oder drücken Sie die Neustart-Taste.

コンピュータを再起動する必要があります。パワーボタンを数秒間押し続けるか、リセットボタンを押してください。

Kernel Architecture – Monolithic

All OS Services in One Kernel Space

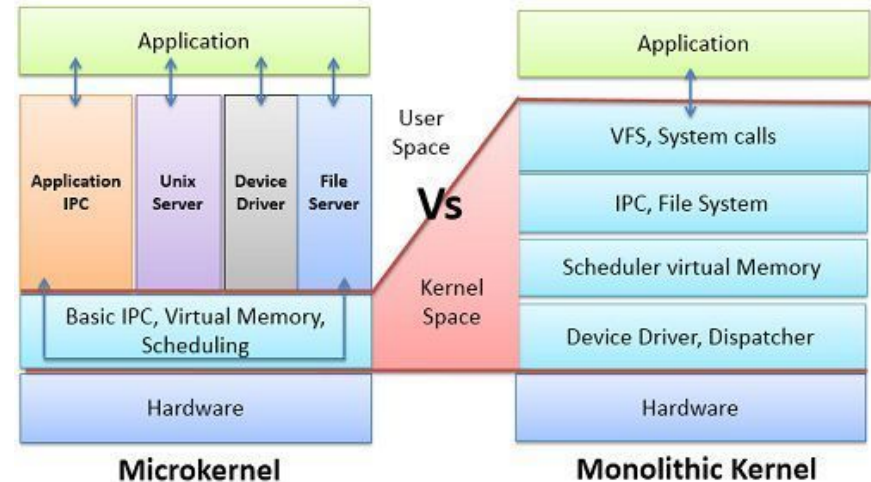
Example: Linux

Advantages:

- Fast (direct function calls)
- Efficient communication between services

Disadvantages:

- One bug can crash entire system
- Large attack surface
- Complexity (30M+ lines of code)



Kernel Architecture – Microkernel

Minimal Kernel, Services in User Space

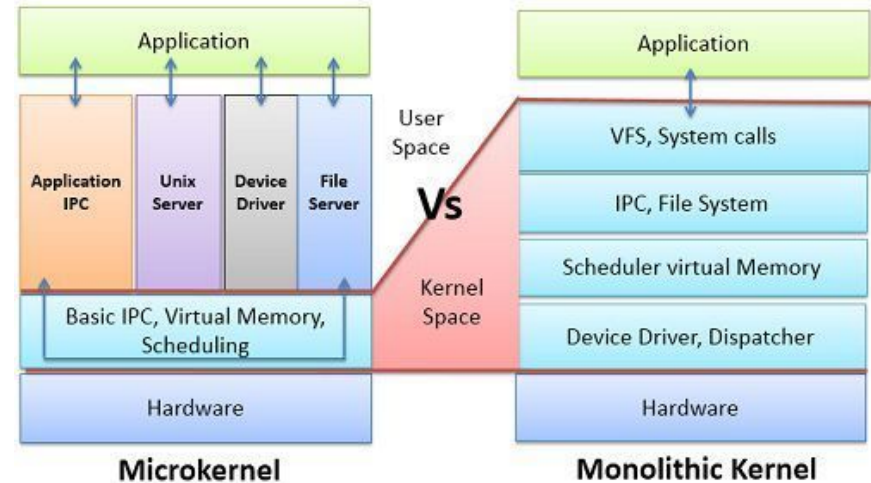
Examples: Original Mach, QNX, MINIX

Kernel provides only:

- Memory management
- IPC (message passing)
- Basic scheduling

Advantages: Robustness, security, modularity

Disadvantages: Performance overhead
(message passing)



macOS XNU – Hybrid Approach

Best of Both Worlds

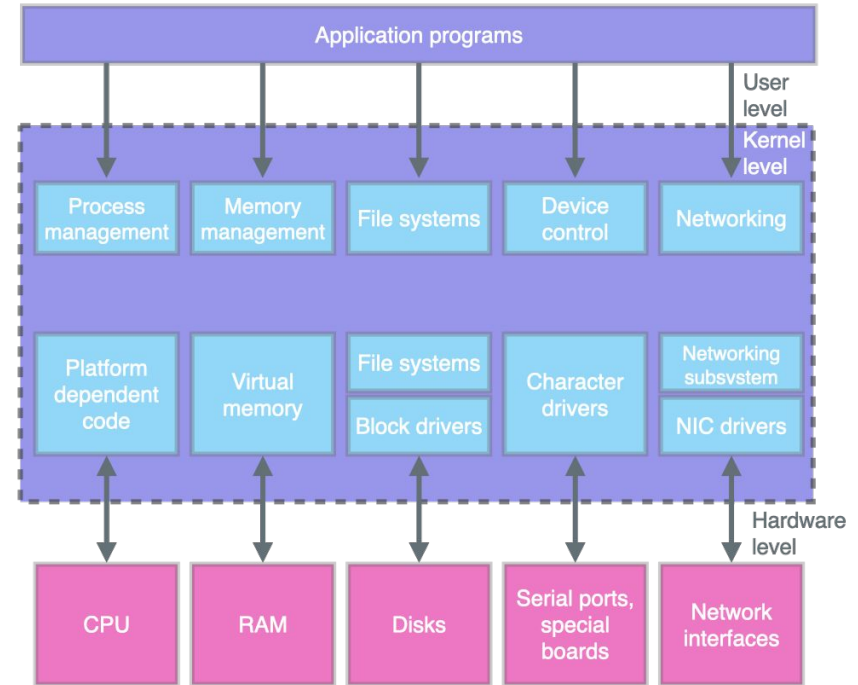
XNU = X is Not Unix

Architecture:

- **Mach microkernel core:** Memory, IPC, threads
- **BSD layer:** POSIX APIs, networking, file systems
- **IOKit:** Device drivers (C++ based)

Result: Microkernel structure + monolithic performance

Your advantage: UNIX-certified, POSIX-compatible



Resources

Required Textbooks:

- Modern Operating Systems (Tanenbaum & Bos)
- Operating System Concepts (Silberschatz et al.)

Tools You'll Use:

- Activity Monitor (process viewing)
- Terminal & command-line tools
- Nand2Tetris software suite (Phase 2)

The Journey Begins

What You've Learned Today:

- What an OS is and why it matters
- Historical evolution to modern systems
- Five core OS services and their coordination
- System calls as the kernel interface
- User vs kernel mode protection
- Monolithic vs microkernel vs hybrid (macOS)

Next Class: Deep dive into processes and threads

- Process creation with fork/exec
- Context switching mechanics



Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.